

Tokyo-Tetsudo-Sim

Comprehensive Technical Documentation & Architecture Report

1. Executive Summary & Project Scope

The **Tokyo Railway Routing Engine** is a high-performance, GPU-accelerated Agent-Based Model (ABM) designed to simulate the daily commutes of 1,000,000 agents across the Greater Tokyo Area in real-time. Built on raw OpenStreetMap (OSM) data, the simulation models individual scheduling, pathfinding, and movement dynamics hour-by-hour.

The primary objective was to mathematically mirror real-world passenger flow, accurately depicting transit density, identifying congestion bottlenecks, and replicating human commuter logic—such as avoiding heavy transfers—at an unprecedented computational scale.

1,000,000

SIMULATED AGENTS

701+

MAPPED TRANSIT HUBS

17 Mins

PROCESSING TIME

2. Phase 1: Foundation & Data Integration

The initial phase involved transforming raw OpenStreetMap (OSM) JSON data into a weighted directed graph. We queried mainline tracks inside the Tokyo boundary using a custom Overpass API script. However, translating geographic data into a contiguous mathematical graph presented severe challenges.

2.1 The Challenge: "Geographical Islands" (Nippori Issue)

Problem: Early prototypes failed to find routes for major hubs like Nippori. The mapped station node (e.g., the ticket gate) was frequently over 110 meters away from the actual railway track nodes. This severed the network, creating "Station Islands" isolated from the tracks.

Avenue Considered & Discarded: Manual hard-coding of links between ticket gates and tracks for every single station. *Why Discarded:* With over 700 stations and thousands of independent track segments, this approach was unscalable and prone to extreme human error during data updates.

Methodology Applied: Spatial Binning and Snapping

We implemented an algorithmic system to group coordinates into geographic buckets using coordinate truncation. By flattening the spatial precision to 3 decimal places, we created buckets of approximately 110 meters. By iterating through local bucket inventories (`wider_spatial_bins`), the engine successfully identified disconnected nodes and built artificial bridge edges, successfully linking isolated station nodes to the nearest functional rail lines.

2.2 The Challenge: Data Noise & "Russian Roulette" Input

Tokyo's major stations had multiple distinct nodes created by different volunteers for different operators (e.g., JR platforms vs. Keisei platforms). The original algorithm would pick one at random when a user selected "Shinjuku," often dropping the agent onto an unlinked or dead-end node.

Methodology Applied: Super-Nodes

The system was rebuilt to group multiple distinct platform IDs into a single virtual locus or "Super-Node," unifying the entire station footprint under one cohesive architectural umbrella.

2.3 Handling Broken Tracks

Human-mapped data contained micro-gaps (5–10 meters) that severed continuous networks like the Yamanote loop. We instituted an **Internal Station Transfers** logic to stitch these tiny gaps within the spatial binning radius, ensuring the mathematical graph remained globally navigable.

3. Phase 2: Graph Architecture & Routing Methodologies

3.1 Combating the "Ghost Track Wormhole" Exploit

Problem: The AI routing engine engaged in "reward hacking." If a track geometry was missing a weight tag in the OSM data, it defaulted to 1 meter. This caused the AI to route millions of agents through impossible geographic wormholes to save time.

Methodology Applied: Haversine Physical Distances

We enforced strict physical distance calculations. For every edge connecting node u and node v , the engine calculates the spherical distance between their latitude and longitude coordinates using the Haversine formula, assigning an immutable weight to every edge.

3.2 Innovation: The 3D "Skyscraper" State-Graph

Standard GPU graph solvers are "line-blind." They only see a 2D map of nodes. Because they do not know what train line an agent is currently on, they cannot charge a time or distance penalty for transferring lines. Consequently, the AI acted like a "tourist with heavy luggage"—sitting on the Yamanote line for 13.5km just to avoid a simple transfer, whereas a real Tokyo commuter would take a 9.1km subway shortcut.

The Solution: We exploded the 2D nodes into a **3D State-Graph Architecture**. Single stations were converted into multi-floor "Skyscrapers" using Tuples. For example, Shinjuku is not a single node, but rather a collection of states: (Shinjuku, Yamanote), (Shinjuku, Chuo), and (Shinjuku, Oedo).

3.3 The Asymmetry Trick for Transfer Penalties

To simulate human preference, we introduced "Staircase" edges between line floors inside the skyscraper hubs, bearing a heavy TRANSFER_PENALTY (default 2,050 meters).

Avenue Considered & Discarded: Charging the penalty equally on both ends of a station visit.
Why Discarded: It penalized agents just for starting their commute.

Adopted Solution: The Asymmetry Trick. Entering a station track from the lobby is cheap (50 units), but exiting to switch lines requires crossing the heavy penalty bridge. This accurately models the friction of navigating large stations like Shinjuku or Tokyo Station.

3.4 Handling Data Anomalies: The "Macron Flip-Flop"

Volunteers tagged identical track segments differently ("Oedo" vs "Öedo"). This caused the console to print endless duplicated transfer instructions. We developed a **Fuzzy Name Matching & Translation Layer** that suppresses utility tracks (HIDDEN_LINES) and standardizes phonetic spellings, ignoring variations unless the operator prefix completely changes.

4. Phase 3: Pathfinding Engine & Performance Optimizations

4.1 The Reversal Trick & Pre-computed Lookup Tables

Avenue Considered & Discarded: Running Single Source Shortest Path (SSSP) algorithms forward (*Start* → *Destination*) dynamically for every agent. *Why discarded?* Calculating Dijkstra for 1,000,000 agents individually per tick would take hours per frame.

Adopted Solution: The Reversal Trick

We reversed all graph edges (*Destination* → *Start*). By running the GPU algorithm backward from each possible destination, we populate a massive global Lookup Table in memory. During runtime, all 1,000,000 agents simply "read" this table simultaneously to find their instant next step, offloading all pathfinding mathematics from the live simulation loop.

4.2 First-Step Propagation over Backtracking

Traditional pathfinding stores a "parent array" and backtracks from the destination to find the route. With 30,000+ nodes, string-based backtracking crippled Python's execution speed. We adopted **First-Step Propagation**, propagating the optimal first step forward instantly during GPU exploration, completely bypassing the need to recursively crawl arrays.

5. Phase 4: GPU-Accelerated Agent Movement Engine

5.1 Memory Architecture (CuPy / cuDF)

To handle a population of one million, object-oriented programming (e.g., an Agent class) was abandoned. The state is strictly maintained in VRAM using **CuPy** matrices and manipulated via **cuDF** (GPU DataFrames).

```
# Column indices mapping for VRAM agent matrix
CURR    = 0  # Current Node ID
WORK    = 1  # Assigned Work Hub
HOME    = 2  # Assigned Home Hub
RAND    = 3  # Random Entropy Seed
TARGET  = 4  # Immediate Pathfinding Target
STATE   = 5  # 0: Idle, 1: Commuting
START   = 6  # Start Time ID
END     = 7  # End Time ID
```

5.2 Time & Shift Triggers (tick() function)

The core simulation operates on a fixed-step tick system. As `tick_time` progresses, it is calculated modulo 24 to simulate hours of the day. Agents are batched into demographic "shifts" based on probabilistic scheduling matrices, dynamically dumping load onto the rail network to create distinct rush-hour spikes.

5.3 System Stability: Data Types and Downcasting

During testing on Linux WSL, the system threw fatal `RuntimeError: Unable to open file` exceptions during Parquet exports. We enforced absolute directory resolution via `os.makedirs`. Furthermore, we implemented dynamic memory protection: arrays are temporarily upgraded to `float64` for calculations to prevent overflow, but explicitly **downcasted back to `int32`** immediately upon cuDF DataFrame compilation.

6. Phase 5: Visualization & Macro-Level Analytics (Kepler.gl)

6.1 Smoothing "Toothpick" Data

Problem: Raw visualization rendered messy, disconnected point-to-point lines resembling fragmented toothpicks. Kepler.gl cannot organically merge broken segments with differing start/end tags.

Solution: We implemented a PySpark aggregation sequence that compiles unlinked coordinate paths and rounds spatial precision to enforce uniformity. This smooths fragmented segments into high-intensity, continuous 3D arcs. This visually uncovers massive macro-bottlenecks, such as the heavy commuter spine stretching from Shinjuku to Tokyo Terminus.

6.2 "God Mode" Metrics & Constraints

Managing the entire simulation purely in VRAM unlocks extreme macro-analytics:

- **Live Congestion Heatmaps:** Calculated globally in one single scatter-add operation using CuPy.
- **Dynamic "Rush Hour" Delays:** Node density is strictly monitored. If an infrastructure node exceeds 5,000 concurrent agents, occupants are mathematically flagged to "lose a tick," perfectly simulating Tokyo's notorious train crush and platform congestion delays.

7. Key Variables & Data Structures Dictionary

`adj_list` / `adjacency_list`

The core 2D topology of the network. Stores relationships as `(distance, line_name)`. Explicitly mapped using integers rather than strings to optimize memory bandwidth across multiprocessor worker threads.

state_edge

A crucial list storing the *reversed* graph topology necessary for the Reversal Trick. Formatted as (Destination_State, Source_State, Distance). It accommodates pure integers for ground tracks or Tuples (e.g., ((303, "Yamanote"), 404, 300.0)) to encode Skyscraper 3D platform jumps.

name_map & station_name_map

Dictionaries translating cryptic OSM Way IDs to human-readable string names, and filtering the massive node list down to the 701 active, physical passenger hubs.

agent_table

The monolithic CuPy matrix housing the life-cycle of every agent. Structured purely in 32-bit integers to remain entirely on the GPU without costly CPU handoffs.

master_track_accumulator

A `np.zeros` integer array allocated exactly once outside the main simulation loop. It tracks global passenger density continuously without triggering Python garbage collection overhead.

8. Key Functions Reference

setup() (data_node_builder.py)

The central initialization sequence. Extracts raw JSON, validates parameters, and returns isolated `station_elements` and `track_elements` strictly to local scopes, bypassing dangerous global variable dependencies.

bin_spatially()

The spatial bridging engine. Truncates precise latitude/longitude floats to bucket and link physically adjacent, but mathematically disconnected, transit nodes.

build_adjacency_list_optimized()

Constructs the network map explicitly leveraging integer IDs. This function was entirely refactored to eliminate string processing, drastically accelerating map generation.

dijkstra_shortest_path()

The transfer-aware routing brain. Deploys the TRANSFER_PENALTY scalar to mathematically penalize heavy line switching and forces paths that align with organic human preference.

tick() (map_sim.py)

The primary physics loop. Translates arbitrary step counts into chronological clock time, triggering probabilistic shift algorithms, and coordinating millions of parallel agent movements across the lookup_table.